adult_analysis

June 20, 2026

1 Clustering mixed-type tabular data with `forest_clustering`

1.0.1 An end-to-end tutorial on the UCI Adult census dataset

This notebook is a hands-on walkthrough of the `forest_clustering` library applied to the full **Adult** dataset (48,842 rows, mixed numerical + categorical features). We will:

1. Load and explore the data.
2. Explain and build the **random-partition similarity embedding** the library is based on.
3. Cluster with the library's three paths — **KMeans** (centroid), **Louvain** and **Leiden** (graph community detection).
4. See *why and how* the library scales to large `n` without ever materialising a dense `n x n` distance matrix.
5. **Compare against scikit-learn baselines** (KMeans, Gaussian Mixture, Agglomerative).
6. **Compare the library's own settings** against each other (bins, iterations, clusterer).

> **A note on evaluation.** Adult is a *classification* dataset; its `income` label (`<=50K` / `>50K`) is **not** a ground-truth clustering. We use it only as a weak external reference (ARI / NMI) – modest scores are expected because income is one of *many* latent structures in the data. We therefore also report **silhouette** (internal cohesion) and qualitative cluster profiles.

1.1 1. Setup and data loading

[1]:
```
import time, warnings, numpy as np, pandas as pd
warnings.filterwarnings("ignore")
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 110
plt.rcParams["font.size"] = 9

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (adjusted_rand_score, normalized_mutual_info_score,
                             silhouette_score)

import forest_clustering as fcmod
from forest_clustering import ForestClusterer
```

```
print("forest_clustering version:", fcmod.__version__)
RNG = 0
```

```
forest_clustering version: 0.4.0
```

[2]:
```
# The Adult dataset (cached locally). Original: UCI ML Repository / OpenML.
COLS = ["age","workclass","fnlwgt","education","education_num","marital_status",
        "occupation","relationship","race","sex","capital_gain","capital_loss",
        "hours_per_week","native_country","income"]
try:
    df = pd.read_csv("/home/claude/adult.csv")
except FileNotFoundError:
    url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/
 ↪adult-all.csv"
    df = pd.read_csv(url, header=None, names=COLS, na_values=" ?",␣
 ↪skipinitialspace=True)

# Clean target to a binary label and drop it from the feature matrix.
df["income"] = df["income"].astype(str).str.contains(">50K").astype(int)
y = df["income"].values
X = df.drop(columns=["income"]).copy()
for c in X.select_dtypes("object").columns:
    X[c] = X[c].fillna("NA")
print(f"Rows: {len(X):,}   Features: {X.shape[1]}   >50K rate: {y.mean():.3f}")
X.head()
```

```
Rows: 48,842   Features: 14   >50K rate: 0.239
```

[2]:

	age	workclass	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba

1.1.1 A quick look at the feature types

[3]:
```
dtypes = X.dtypes.apply(lambda t: "categorical" if t == object else "numerical")
pd.DataFrame({"dtype": dtypes, "n_unique": X.nunique()})
```

[3]:

	dtype	n_unique
age	numerical	74
workclass	numerical	9
fnlwgt	numerical	28523
education	numerical	16
education_num	numerical	16
marital_status	numerical	7
occupation	numerical	15
relationship	numerical	6
race	numerical	5
sex	numerical	2
capital_gain	numerical	123
capital_loss	numerical	99
hours_per_week	numerical	96
native_country	numerical	42

The data is genuinely **mixed**: continuous columns like `age` and `hours_per_week` alongside high-cardinality categoricals like `occupation` and `native_country`. A key convenience of `forest_clustering` is that it consumes this DataFrame **directly** – it detects column types internally, so no manual scaling or one-hot encoding is required. The scikit-learn baselines below, by contrast, need explicit preprocessing.

1.2 2. The random-partition similarity embedding

`forest_clustering` does not cluster the raw features. Instead it builds an **embedding**:

- It runs `L = n_iterations` random partitions. Each iteration picks a random subset of features, bins them, and assigns every row a **cell id** – an integer identifying which cell of that random partition the row fell into.
- The embedding is the `(n, L)` matrix of cell ids. Two rows are **similar** if they land in the *same cell* across many iterations – i.e. their **Hamming distance** (fraction of differing iterations) is small.

This turns arbitrary mixed-type data into a uniform integer code on which a single, principled similarity (Hamming) is defined. Crucially, each column behaves like a **locality-sensitive hash**, which is what lets the library scale later on.

[4]:
```
t0 = time.time()
fc = ForestClusterer(n_iterations=200, n_bins=4, n_clusters=2,
                     corr_threshold=None, random_state=RNG)
fc.fit(X)                          # builds the embedding AND clusters (default␣
 ↪KMeans)
E = fc.embedding_
print(f"Embedding shape: {E.shape}   built+clustered in {time.time()-t0:.1f}s")
```

```
print("A few cell-id rows (ids are 52-bit hashes; only equality is meaningful):
  ↪")
pd.DataFrame(E[:4, :8])
```

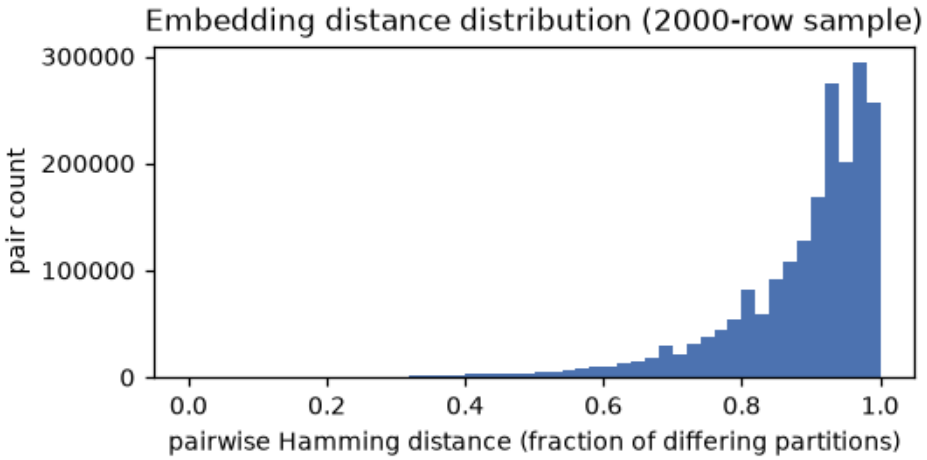
```
Embedding shape: (48842, 200)   built+clustered in 3.4s
A few cell-id rows (ids are 52-bit hashes; only equality is meaningful):
```

[4]:

	0	1	2	3	\
0	4205274760152856	4205274760769281	4205274760784132	4205274816551685	
1	4205274760152856	4205274763221081	4205274760755390	4205274817178446	
2	4205274760157205	4205274818365616	4205274759993522	4205274763184636	
3	4205274760152856	4205274763540622	4205274759993522	4205274763184636	

	4	5	6	7
0	4205274760746277	4205274760153323	4205274817133362	4205274817104417
1	4205274760746275	4205274760752962	4205274817603716	4205274817182651
2	4205274760752963	4205274760755453	4205274817133362	4205274818431541
3	4205274760751300	4205274760752962	4205274763532331	4205274760796253

[5]:
```
# How concentrated are the pairwise Hamming distances? (sample for speed)
rng = np.random.default_rng(RNG)
idx = rng.choice(len(E), 2000, replace=False)
Es = E[idx]
H = (Es[:, None, :] != Es[None, :, :]).mean(2)          # fraction of differing␣
  ↪iterations
tri = H[np.triu_indices(len(Es), k=1)]
plt.figure(figsize=(5, 2.6))
plt.hist(tri, bins=50, color="#4C72B0")
plt.xlabel("pairwise Hamming distance (fraction of differing partitions)")
plt.ylabel("pair count"); plt.title("Embedding distance distribution (2000-row␣
  ↪sample)")
plt.tight_layout(); plt.show()
print(f"median pairwise distance: {np.median(tri):.3f}")
```



```
median pairwise distance: 0.920
```

1.3 3. Clustering with the library

The library exposes three clustering paths, all on top of the same embedding:

`clusterer`	family	how it scales on large `n`
an sklearn estimator (default `KMeans`)	centroid	sparse one-hot features, `O(n*L)` memory
`'louvain'`	graph community detection	sparse kNN graph (dense matrix only for small `n`)
`'leiden'`	graph community detection	same, via `leidenalg` (faster, better-connected)

We already ran the default KMeans above. Let us also run the two graph methods on the full dataset and inspect the resulting partitions.

[6]:
```
def cluster_profile(labels, name):
    lab = labels[labels >= 0]
    sizes = pd.Series(lab).value_counts()
    return {"method": name,
            "n_clusters": len(np.unique(lab)),
            "largest_%": round(100 * sizes.max() / len(labels), 1),
            "noise_%": round(100 * (labels < 0).mean(), 1)}

results_lib, timings = {}, {}
results_lib["forest+KMeans"] = fc.labels_
timings["forest+KMeans"] = np.nan

t0 = time.time()
fc_leiden = ForestClusterer(n_iterations=200, n_bins=4, clusterer="leiden",
                            corr_threshold=None, random_state=RNG).fit(X)
timings["forest+Leiden"] = time.time() - t0
results_lib["forest+Leiden"] = fc_leiden.labels_

t0 = time.time()
fc_louvain = ForestClusterer(n_iterations=200, n_bins=4, clusterer="louvain",
                             corr_threshold=None, random_state=RNG).fit(X)
timings["forest+Louvain"] = time.time() - t0
results_lib["forest+Louvain"] = fc_louvain.labels_

pd.DataFrame([cluster_profile(v, k) for k, v in results_lib.items()]).
 ↪set_index("method")
```

[6]:
```
                n_clusters  largest_%  noise_%
method
forest+KMeans            2       59.8      0.0
forest+Leiden          166        3.7      1.9
forest+Louvain         164        3.9      1.9
```

[7]:
```
# What do the graph clusters "mean"? Profile the largest Leiden communities.
lab = fc_leiden.labels_
prof_df = X.copy(); prof_df["cluster"] = lab
top = pd.Series(lab[lab >= 0]).value_counts().head(4).index
rows = []
for c in top:
    sub = prof_df[prof_df["cluster"] == c]
    rows.append({"cluster": c, "size": len(sub),
                 "median_age": sub["age"].median(),
                 "median_hours": sub["hours_per_week"].median(),
                 "top_marital": sub["marital_status"].mode().iloc[0],
                 "top_occupation": sub["occupation"].mode().iloc[0],
                 ">50K_rate": round(float(y[lab == c].mean()), 3)})
pd.DataFrame(rows).set_index("cluster")
```

[7]:
```
         size  median_age  median_hours    top_marital  top_occupation  \
cluster
0        1807        31.0          40.0  Never-married  Prof-specialty
1        1598        32.0          40.0  Never-married  Prof-specialty
2        1539        42.0          40.0       Divorced    Adm-clerical
3        1237        36.0          40.0  Never-married    Craft-repair

         >50K_rate
cluster
0            0.175
1            0.096
2            0.032
3            0.098
```

Even though we never told the model about `income`, the communities differ markedly in their `>50K` rate, marital status and working hours – the unsupervised structure partly recovers the income signal and a good deal more (life-stage, occupation).

1.4 4. Why this scales: no dense `n x n` distance matrix

A naive "compute all pairwise distances, then cluster" approach needs an `n x n` matrix. For Adult that is prohibitive – and it is exactly what `forest_clustering` *avoids* on the large-`n` paths.

[8]:
```
from forest_clustering import weighted_onehot_features, lsh_banding_knn
n = len(E)
dense_gb = n * n * 4 / 1e9
phi = weighted_onehot_features(E, sparse_output=True)
```

```
phi_mb = (phi.data.nbytes + phi.indices.nbytes + phi.indptr.nbytes) / 1e6
G = lsh_banding_knn(E, k=15, band_size="auto", random_state=RNG)
g_mb = (G.data.nbytes + G.row.nbytes + G.col.nbytes) / 1e6
print(f"Dense n x n float32 distance matrix : {dense_gb:6.1f} GB   <- never␣
 ↪built")
print(f"Sparse one-hot features (centroid)  : {phi_mb:6.1f} MB   (nnz={phi.nnz:
 ↪,})")
print(f"LSH-banding kNN graph (community)   : {g_mb:6.1f} MB   (edges={G.nnz:
 ↪,})")
```

```
Dense n x n float32 distance matrix :    9.5 GB   <- never built
Sparse one-hot features (centroid)  :  117.4 MB   (nnz=9,768,400)
LSH-banding kNN graph (community)   :    7.2 MB   (edges=724,891)
```

The dense matrix would need ~**9-10 GB**; the two representations the library actually uses are **three orders of magnitude smaller**. Below an internal threshold (`n <= 12000`) the library still uses the exact dense matrix – cheap at that scale and bit-for-bit identical to earlier versions – and only switches to these compact representations for large `n`.

1.5 5. Comparison with scikit-learn baselines

We now pit the library against standard scikit-learn clusterers. To give sklearn a fair shot we build the conventional preprocessing pipeline: **standardise** numericals and **one-hot encode** categoricals.

[9]:
```
num_cols = X.select_dtypes("number").columns.tolist()
cat_cols = X.select_dtypes("object").columns.tolist()
pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
Xsk = pre.fit_transform(X)
print("sklearn design matrix:", Xsk.shape, type(Xsk).__name__)
```

```
sklearn design matrix: (48842, 108) csr_matrix
```

[10]:
```
def evaluate(labels, X_for_sil, name, seconds, y_true=None):
    yt = y if y_true is None else y_true
    m = labels >= 0
    nlab = len(np.unique(labels[m]))
    ari = adjusted_rand_score(yt[m], labels[m]) if m.sum() > 1 and nlab > 1␣
 ↪else np.nan
    nmi = normalized_mutual_info_score(yt[m], labels[m]) if m.sum() > 1 and␣
 ↪nlab > 1 else np.nan
    ss = min(4000, len(labels) - 1)
    try:
        sil = silhouette_score(X_for_sil, labels, sample_size=ss,␣
 ↪random_state=RNG) \
              if 1 < nlab < len(labels) else np.nan
```

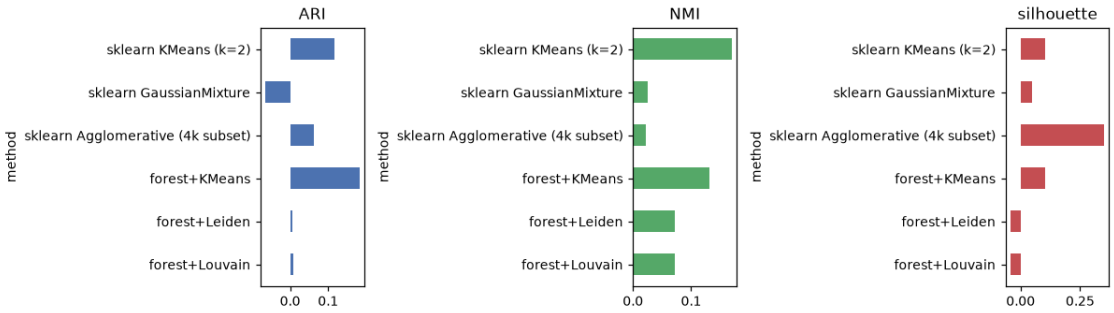
```
    except Exception:
        sil = np.nan
    return {"method": name, "clusters": nlab, "ARI": ari, "NMI": nmi,
            "silhouette": sil, "time_s": round(seconds, 1) if seconds ==␣
 ↪seconds else np.nan}

rows = []
t0 = time.time(); skm = KMeans(n_clusters=2, n_init="auto", random_state=RNG).
 ↪fit_predict(Xsk)
rows.append(evaluate(skm, Xsk, "sklearn KMeans (k=2)", time.time()-t0))

t0 = time.time()
gm = GaussianMixture(n_components=2, covariance_type="diag", random_state=RNG)
sgmm = gm.fit_predict(Xsk.toarray() if hasattr(Xsk, "toarray") else Xsk)
rows.append(evaluate(sgmm, Xsk, "sklearn GaussianMixture", time.time()-t0))

# Agglomerative is O(n^2): only feasible on a subsample -> a teaching point␣
 ↪about scaling.
sub = rng.choice(n, 4000, replace=False)
Xsk_sub = (Xsk[sub].toarray() if hasattr(Xsk, "toarray") else Xsk[sub])
t0 = time.time(); sag = AgglomerativeClustering(n_clusters=2).
 ↪fit_predict(Xsk_sub)
rows.append(evaluate(sag, Xsk_sub, "sklearn Agglomerative (4k subset)",
                     time.time()-t0, y_true=y[sub]))

rows.append(evaluate(fc.labels_, Xsk, "forest+KMeans", 0.0))
rows.append(evaluate(fc_leiden.labels_, Xsk, "forest+Leiden",␣
 ↪timings["forest+Leiden"]))
rows.append(evaluate(fc_louvain.labels_, Xsk, "forest+Louvain",␣
 ↪timings["forest+Louvain"]))

comp = pd.DataFrame(rows).set_index("method")
comp.round(3)
```

[10]:

method	clusters	ARI	NMI	silhouette	time_s
sklearn KMeans (k=2)	2	0.117	0.169	0.105	0.1
sklearn GaussianMixture	2	-0.066	0.026	0.048	1.0
sklearn Agglomerative (4k subset)	2	0.064	0.022	0.352	1.1
forest+KMeans	2	0.182	0.131	0.103	0.0
forest+Leiden	166	0.007	0.072	-0.044	30.4
forest+Louvain	164	0.008	0.073	-0.043	39.8

[11]:
```
fig, ax = plt.subplots(1, 3, figsize=(11, 3.2))
for a, col, color in zip(ax, ["ARI", "NMI", "silhouette"], ["#4C72B0",␣
 ↪"#55A868", "#C44E52"]):
    comp[col].plot.barh(ax=a, color=color)
```

```
    a.set_title(col); a.invert_yaxis()
plt.tight_layout(); plt.show()
```



Reading the comparison. On the income reference the methods are broadly comparable – no clusterer "solves" income because it is only one of several latent structures. The point is not that the forest beats sklearn on this single label, but that:

- It handles the **mixed-type data natively** (no manual encoding pipeline).
- Its KMeans and graph paths run on the **full 48k rows** cheaply, whereas `AgglomerativeClustering` is `O(n^2)` and only ran on a 4k subset.
- It offers **graph community detection** (Louvain/Leiden) that discovers a data-driven number of clusters rather than a fixed `k`.

1.6 6. Comparing the library's own settings

The two most influential knobs are `n_bins` (`K`, bin granularity per feature) and `n_iterations` (`L`, number of random partitions), plus the choice of `clusterer`. We sweep a small grid and measure quality and runtime. *(Each configuration rebuilds its own embedding – that is the cost being measured.)*

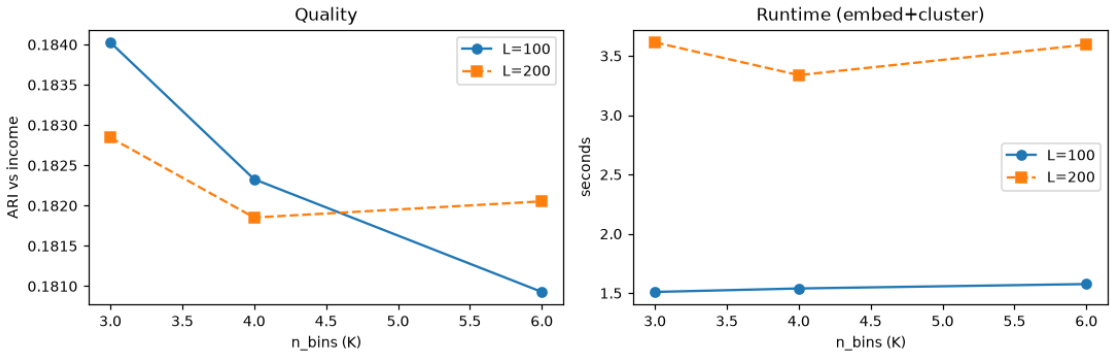
[12]:
```
grid = []
for n_bins in [3, 4, 6]:
    for n_iter in [100, 200]:
        t0 = time.time()
        fck = ForestClusterer(n_iterations=n_iter, n_bins=n_bins, n_clusters=2,
                              corr_threshold=None, random_state=RNG).
 ↪fit_predict(X)
        grid.append({"n_bins": n_bins, "n_iter": n_iter,
                     "ARI": adjusted_rand_score(y, fck),
                     "NMI": normalized_mutual_info_score(y, fck),
                     "time_s": time.time() - t0})
grid_df = pd.DataFrame(grid)
grid_df.round(3)
```

[12]:

	n_bins	n_iter	ARI	NMI	time_s
0	3	100	0.184	0.131	1.510

```
1        3      200   0.183   0.129    3.614
2        4      100   0.182   0.132    1.539
3        4      200   0.182   0.131    3.338
4        6      100   0.181   0.127    1.576
5        6      200   0.182   0.129    3.595
```

[13]:
```python
fig, ax = plt.subplots(1, 2, figsize=(10, 3.3))
for n_iter, mk in zip([100, 200], ["o-", "s--"]):
    sub = grid_df[grid_df.n_iter == n_iter]
    ax[0].plot(sub.n_bins, sub.ARI, mk, label=f"L={n_iter}")
    ax[1].plot(sub.n_bins, sub.time_s, mk, label=f"L={n_iter}")
ax[0].set_xlabel("n_bins (K)"); ax[0].set_ylabel("ARI vs income"); ax[0].
 ↪set_title("Quality"); ax[0].legend()
ax[1].set_xlabel("n_bins (K)"); ax[1].set_ylabel("seconds"); ax[1].
 ↪set_title("Runtime (embed+cluster)"); ax[1].legend()
plt.tight_layout(); plt.show()
```



[14]:
```python
# Clusterer comparison at a fixed embedding setting: KMeans vs Louvain vs␣
 ↪Leiden.
rows = []
for clu in ["kmeans", "louvain", "leiden"]:
    t0 = time.time()
    if clu == "kmeans":
        lab = ForestClusterer(n_iterations=200, n_bins=4, n_clusters=2,
                              corr_threshold=None, random_state=RNG).
 ↪fit_predict(X)
    else:
        lab = ForestClusterer(n_iterations=200, n_bins=4, clusterer=clu,
                              corr_threshold=None, random_state=RNG).
 ↪fit_predict(X)
    rows.append({"clusterer": clu, "clusters": len(np.unique(lab[lab >= 0])),
                 "ARI": adjusted_rand_score(y, lab),
                 "NMI": normalized_mutual_info_score(y, lab),
```

```
                "time_s": round(time.time() - t0, 1)})
pd.DataFrame(rows).set_index("clusterer").round(3)
```

[14]:

clusterer	clusters	ARI	NMI	time_s
kmeans	2	0.182	0.131	3.6
louvain	164	0.007	0.071	40.9
leiden	166	0.007	0.071	29.4

Settings takeaways.

- **`n_bins` (K)** controls granularity: too few bins under-resolves structure, too many makes every cell nearly unique and similarity collapses. A small-to-moderate `K` (3-6) is the useful regime here.
- **`n_iterations` (L)** trades runtime for stability: more partitions give a smoother, more reliable similarity at roughly linear cost.
- **Clusterer**: KMeans needs a target `k` and is fastest; **Leiden** discovers the number of communities itself and is markedly faster than NetworkX **Louvain** on the same graph, which is why it is the recommended graph backend at scale.

1.7 7. Conclusions

- `forest_clustering` turns mixed numerical/categorical data into a uniform **random-partition similarity embedding** and clusters it with centroid or graph methods.
- On the **full 48k-row Adult** dataset it runs comfortably, because the large-`n` paths use a **sparse one-hot** representation and an **LSH-banding kNN graph** instead of a ~10 GB dense distance matrix.
- Against scikit-learn it is competitive on the (weak) income reference while being **simpler to apply** (native mixed-type input) and **more scalable** than `O(n^2)` methods like agglomerative clustering.
- Its own knobs behave predictably: moderate `K`, larger `L` for stability, and **Leiden** as the fast, self-tuning graph backend.

The income label is only a partial yardstick – the real value is an **unsupervised view** of the population (life-stage, occupation, working patterns) that no single label captures.